HANDBOOK

Junior Software Developer

Grade XII – IT-ITeS Vocational Education

RDBMS • Advanced Python • Software Engineering • Emerging Technologies

A structured textbook for database practice, software development and responsible technology use.

Prepared as a learning-focused companion based on the supplied Junior Software Developer Grade XII study material.

Contents

1 Relational Database Foundations **1**
1.1 Data and information 1
1.2 File systems and databases 1
1.3 Tables, keys and constraints 1
1.4 Database models 2

2 SQL: Defining and Manipulating Data **3**
2.1 SQL command families 3
2.2 Create, insert and retrieve 3
2.3 Joins and transactions 4

3 SQL Functions and Relational Design **5**

4 Python Data Structures: Stack and Queue **6**

5 Exception Handling in Python **7**

6 File Handling in Python **8**

7 NumPy Arrays **9**

8 Pandas and Series **10**

9 Visualisation with Matplotlib **11**

10 Connecting Python with MySQL **12**

11 Software Engineering Concepts **13**

12 Software Development Life Cycle **14**

13 Software Testing and Maintenance **15**

14 Emerging Technologies **16**

15 Social Impact, Ethics and Cyber Safety **18**

16 Integrated Practical Project **19**

Chapter 1

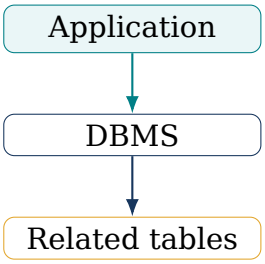
Relational Database Foundations

1.1 Data and information

Data are recorded facts such as student numbers, prices, dates or quantities. Information is data arranged to answer a question. A database is an organised collection of related data that can be searched and updated consistently. A database management system (DBMS) defines, stores, retrieves and protects data. A relational DBMS represents data through tables of rows and columns.

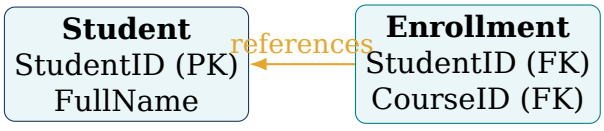
1.2 File systems and databases

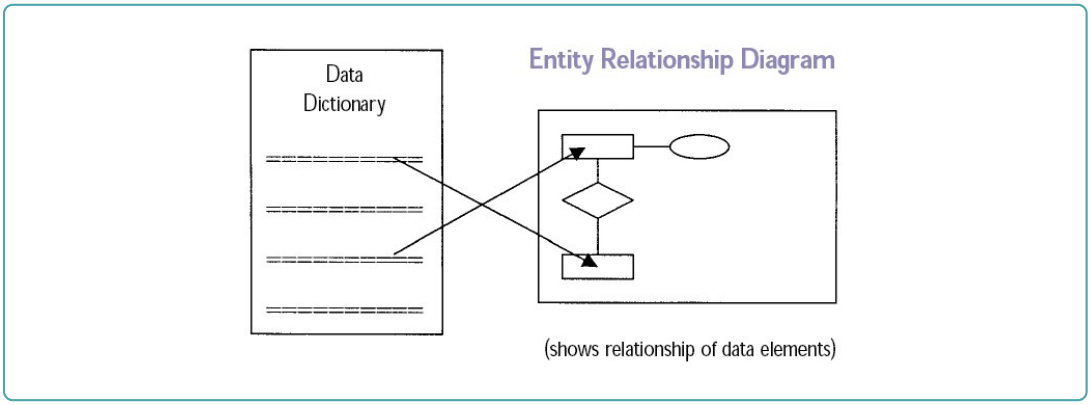
Independent files may duplicate values and become inconsistent when only some copies are updated. A DBMS supports controlled access, validation, shared data, backup and query-based retrieval. These benefits depend on sound design and configuration.



1.3 Tables, keys and constraints

A relation is a table; a tuple is a row; an attribute is a column; a domain is the set of valid values; a schema is the planned structure; an instance is the data present at a particular time. A candidate key uniquely identifies a row. One candidate becomes the primary key. A composite key combines attributes. A foreign key references a key in another table. Constraints such as NOT NULL, UNIQUE and CHECK express rules close to the data.





An ER diagram maps entities and the relationships that become tables and key links in a relational database. Source and licence are listed in IMAGE-CREDITS.txt.

1.4 Database models

Relational databases use related tables. Document stores hold document-like records; key-value stores map keys to values; graph databases emphasise relationships. Select a model based on data shape, consistency needs and query patterns.

Review Questions

1. Explain data versus information.
2. Give four limitations of independent file-based records.
3. Define relation, tuple, attribute, schema and instance.
4. Distinguish primary and foreign keys.
5. Give one relational database use case.

Chapter 2

SQL: Defining and Manipulating Data

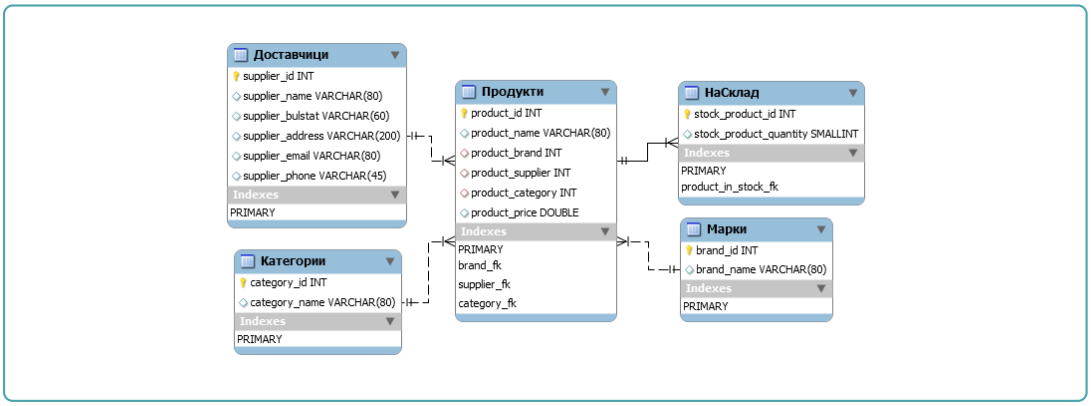
2.1 SQL command families

SQL defines database structures, changes records, retrieves information and controls access. DDL includes CREATE, ALTER and DROP; DML includes INSERT, UPDATE and DELETE; SELECT retrieves records; DCL manages permissions; TCL includes COMMIT and ROLLBACK.

2.2 Create, insert and retrieve

```
CREATE DATABASE training;
USE training;
CREATE TABLE Student (
  StudentID INT PRIMARY KEY,
  FullName VARCHAR(80) NOT NULL,
  ClassLevel TINYINT NOT NULL,
  Email VARCHAR(120) UNIQUE
);
INSERT INTO Student (StudentID, FullName, ClassLevel)
VALUES (1204, 'Anu Rao', 12);
SELECT StudentID, FullName
FROM Student
WHERE ClassLevel = 12
ORDER BY FullName;
```

Choose types deliberately. Use date types for dates, suitable text types for names and exact decimal types for currency. WHERE filters rows; ORDER BY sorts; GROUP BY forms groups; HAVING filters groups after aggregation.



The diagram illustrates how a database design organises related records and supports consistent retrieval. Source and licence are listed in IMAGE-CREDITS.txt.

```
SELECT ClassLevel, COUNT(*) AS StudentCount
FROM Student
GROUP BY ClassLevel
HAVING COUNT(*) >= 5
ORDER BY StudentCount DESC;
```

2.3 Joins and transactions

An inner join returns matching rows; a left join retains all rows from the left table. A transaction groups changes into one logical unit. ACID describes atomicity, consistency, isolation and durability. Check the condition of UPDATE or DELETE with SELECT first. Use parameterised queries in applications instead of concatenating untrusted input.



A code editor is a practical place to inspect SQL and application logic. Image source and licence are listed in IMAGE-CREDITS.txt.

Review Questions

1. Explain DDL and DML with examples.
2. Write a query to retrieve Grade XII students alphabetically.
3. Distinguish WHERE and HAVING.
4. What is a join?
5. Why are parameterised queries important?

Chapter 3

SQL Functions and Relational Design

Single-row functions transform values per row. String functions change case or measure text; numeric functions round values; date functions work with calendar values. Aggregate functions such as COUNT, SUM, AVG, MIN and MAX summarise groups. COUNT(column) generally excludes null values, while COUNT(*) counts rows.

```
SELECT UPPER(FullName) AS DisplayName,
       CHAR_LENGTH(FullName) AS NameLength
FROM Student;
```

Normalisation reduces duplication and update anomalies. First normal form expects atomic values and no repeating groups. Second normal form removes partial dependencies on part of a composite key. Third normal form removes inappropriate transitive dependencies among non-key attributes.

If an order table repeats a supplier address on every row, corrections require many updates. A separate Supplier table referenced by a key reduces duplication.

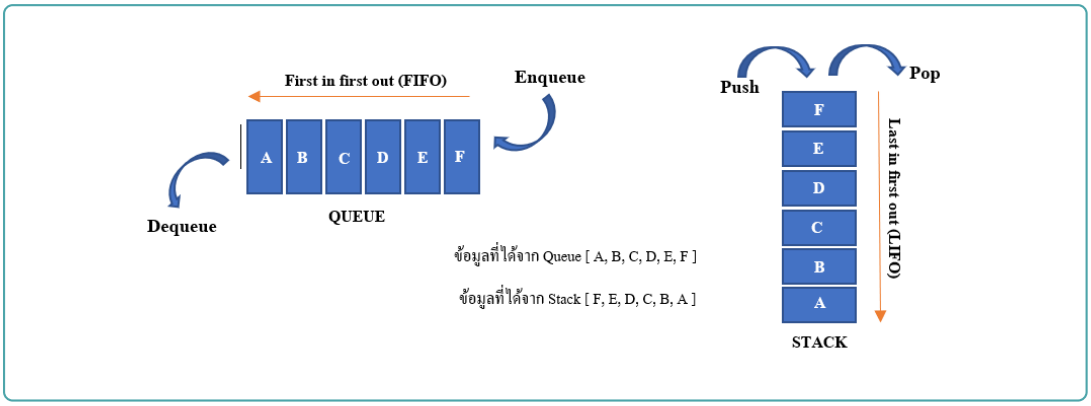
Review Questions

1. Distinguish single-row and aggregate functions.
2. Why can COUNT(*) differ from COUNT(Email)?
3. What is an update anomaly?
4. Explain normalisation.
5. List the main database design steps.

Chapter 4

Python Data Structures: Stack and Queue

A stack follows last-in, first-out (LIFO); the last item added is removed first. A queue follows first-in, first-out (FIFO); the earliest item is removed first. Stacks suit undo histories and function calls; queues suit print jobs and service requests.



The structures differ in the order items leave the collection. Source and licence are listed in IMAGE-CREDITS.txt.

```
from collections import deque
stack = []
stack.append("first")
stack.append("second")
print(stack.pop())        # second
queue = deque(["first", "second"])
queue.append("third")
print(queue.popleft())    # first
```

Use deque for efficient queue operations. Test empty structures and decide how to handle attempts to remove an item when none exists.

Review Questions

1. Define LIFO and FIFO.
2. Give two uses of each structure.
3. Why is deque useful for queues?
4. How should an empty-stack pop be handled?

Chapter 5

Exception Handling in Python

A syntax error prevents parsing; a runtime exception occurs during execution; a logic error produces an incorrect result even though the program runs. Use try for risky operations, except for expected exception types, else for success-only actions and finally for cleanup.

```
raw = input("Enter a whole number: ")
try:
    value = int(raw)
except ValueError:
    print("Please enter a valid whole number.")
else:
    print("Double:", value * 2)
finally:
    print("Input attempt finished.")
```

Catch the narrowest useful exception. Avoid broad handlers that hide defects. Raise an exception when an input violates a function's documented contract.

Review Questions

1. Distinguish syntax, runtime and logic errors.
2. Explain else and finally.
3. Why avoid bare except clauses?
4. Write an example handling ValueError.

Chapter 6

File Handling in Python

Text files store characters; binary files store bytes. Modes include r for reading, w for replacing content, a for appending and x for exclusive creation. The with statement closes files reliably after a block.

```
with open("notes.txt", "w", encoding="utf-8") as file:
    file.write("Python file handling\n")
with open("notes.txt", "r", encoding="utf-8") as file:
    print(file.read())
```

CSV stores rows and columns; Python's csv module handles quoted values. JSON represents objects, arrays and primitive values and is commonly used by APIs. Validate user-provided files, avoid overwriting valuable data without backups and never commit secrets to source control.

```
import csv
with open("marks.csv", newline="", encoding="utf-8") as file:
    for row in csv.DictReader(file):
        print(row.get("StudentName"), row.get("Marks"))
```

Review Questions

1. Distinguish w and a modes.
2. Why is with open preferred?
3. Why use the csv module?
4. Give two file-safety precautions.

Chapter 7

NumPy Arrays

NumPy supports multidimensional arrays and numerical operations. Arrays usually contain values of a common type and support vectorised calculations.

```
import numpy as np
marks = np.array([72, 81, 69, 94])
print(marks.mean())
print(marks.max())
print(marks[1:3])
print(marks.shape, marks.ndim, marks.dtype)
```

Shape describes dimensions, ndim gives the number of dimensions and dtype reports the element type. Vectorisation applies operations to whole arrays. Broadcasting allows compatible shapes to work together; check shapes to avoid unintended calculations.

Review Questions

1. Compare lists and NumPy arrays.
2. Explain shape, ndim and dtype.
3. What is vectorisation?
4. Why check array shapes?

Chapter 8

Pandas and Series

Pandas provides labelled structures for cleaning and analysing tabular data. A Series is a one-dimensional labelled sequence; a DataFrame is a table-like structure.

```
import pandas as pd
data = pd.DataFrame({
    "student": ["Asha", "Ravi", "Meena"],
    "marks": [82, 76, 91]
})
print(data.head())
print(data["marks"].mean())
print(data.describe())
```

Use column names to select values and Boolean conditions to filter rows. Inspect missing values with isna(); remove or fill them only when justified. Grouping summarises categories. Before merging, check key uniqueness because many-to-many joins may multiply rows.

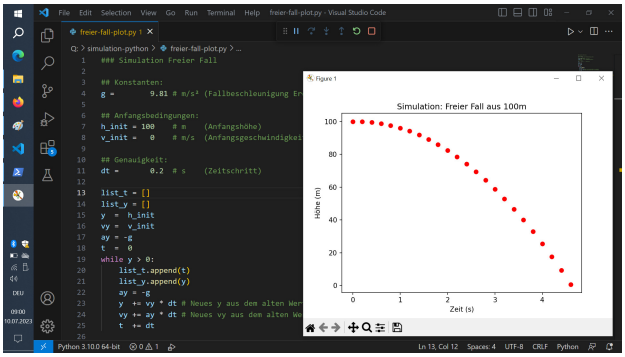
Review Questions

1. Distinguish Series and DataFrame.
2. How can missing values be inspected?
3. Why might a merge increase row count?
4. Describe a simple marks analysis.

Chapter 9

Visualisation with Matplotlib

Bar charts compare categories; line charts show change over an ordered scale; scatter plots explore numeric relationships; histograms show distributions.



A programming environment can connect calculations with visual interpretation; see IMAGE-CREDITS.txt.

```
import matplotlib.pyplot as plt
subjects = ["SQL", "Python", "Networks"]
scores = [78, 86, 74]
plt.bar(subjects, scores)
plt.title("Practice assessment scores")
plt.xlabel("Subject")
plt.ylabel("Score")
plt.ylim(0, 100)
plt.tight_layout()
plt.show()
```

Label axes and units, use readable scales and explain the data source. Avoid decoration that distorts comparisons or implies unsupported precision.

Review Questions

1. Match chart types to questions.
2. Why label axes?
3. Name two ways charts can mislead.
4. Outline steps to create a bar chart.

Chapter 10

Connecting Python with MySQL

A database connector lets Python send SQL statements and receive results. A typical workflow loads configuration securely, opens a connection, creates a cursor, executes parameterised statements, commits successful changes, rolls back failures and closes resources.

```
# Illustrative pattern; configure a MySQL connector first.
sql = "SELECT StudentID, FullName FROM Student WHERE ClassLevel = %s"
cursor.execute(sql, (12,))
rows = cursor.fetchall()
for row in rows:
    print(row)
```

Placeholder syntax depends on the connector. Keep credentials outside source code, grant least privilege, handle network failures and avoid exposing sensitive records in logs.

Review Questions

1. Describe the Python-to-MySQL workflow.
2. Why use parameterised statements?
3. Explain commit and rollback.
4. Give three ways to protect credentials.

Chapter 11

Software Engineering Concepts

Software engineering applies systematic methods to build, test, release and maintain software. Functional requirements describe actions the system must perform. Non-functional requirements describe qualities such as response time, security, availability and accessibility. Requirements should be testable and unambiguous. Modularity divides a system into components with clear responsibilities. High cohesion means a component's tasks belong together; low coupling limits unnecessary dependencies. Quality attributes include reliability, maintainability, portability, scalability, usability and security.

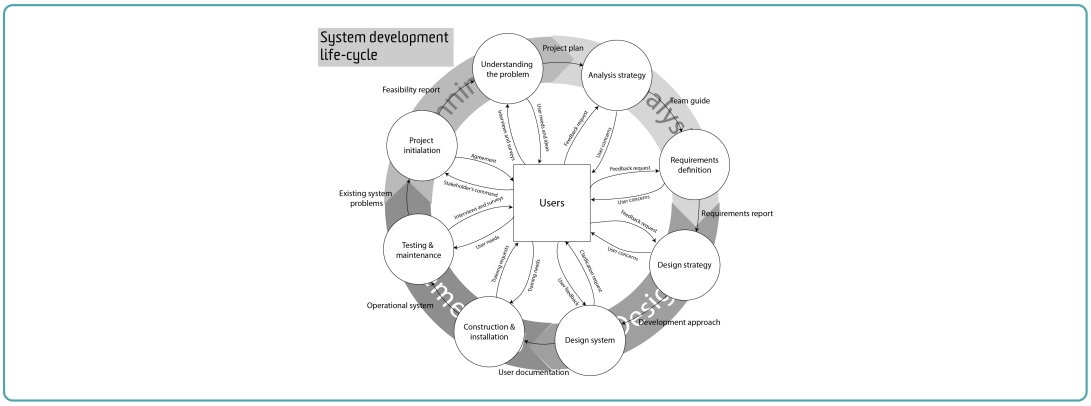
Review Questions

1. Distinguish functional and non-functional requirements.
2. Define cohesion and coupling.
3. List five quality attributes.
4. Rewrite a vague requirement measurably.

Chapter 12

Software Development Life Cycle

Typical activities include planning, requirements analysis, design, implementation, testing, deployment and maintenance. Sequential and iterative approaches organise these activities differently; feedback may send work back to earlier stages.



A published SDLC diagram illustrates the stages from planning and requirements through design, implementation, testing, deployment and maintenance. Image by John Yeung, Wikimedia Commons, CC BY-SA 3.0.

Plan-driven methods suit some stable or constrained projects. Iterative methods deliver increments and incorporate feedback. Version control records changes; meaningful commits and code review support collaboration. Risk planning records likelihood, impact, owners and mitigation actions.

Review Questions

1. List typical SDLC activities.
2. Compare plan-driven and iterative approaches.
3. Why is code review useful?
4. Describe a project risk and mitigation.

Chapter 13

Software Testing and Maintenance

Unit tests check small components; integration tests check interactions; system tests assess the full product; acceptance tests assess user needs. Regression tests check that a change has not broken existing behaviour. A test case defines inputs, preconditions, actions and expected results. Include typical values, boundaries, invalid inputs and failure conditions. Debug systematically: reproduce the issue, gather evidence, narrow the cause, make a fix and rerun relevant tests. Deployment planning includes configuration, database migration, monitoring and rollback.

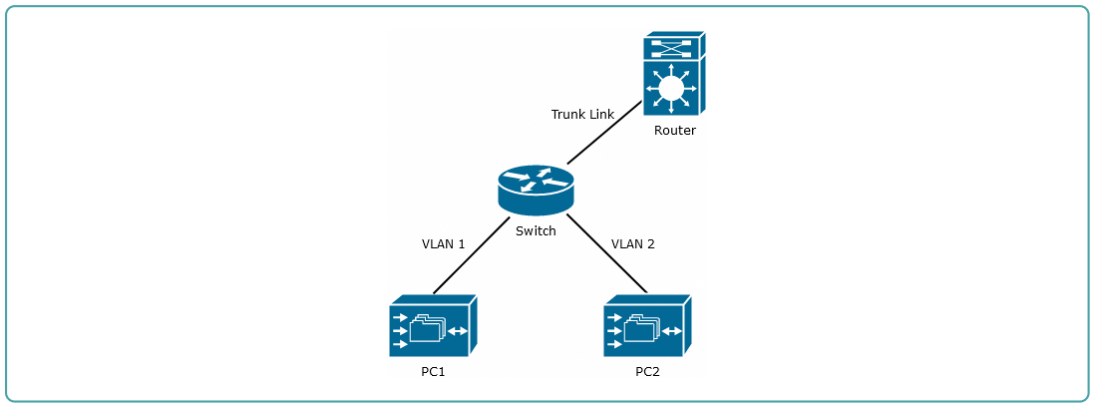
Review Questions

1. Explain unit, integration, system and acceptance testing.
2. What is regression testing?
3. Why test boundary values?
4. Outline a safe debugging process.
5. What belongs in a rollback plan?

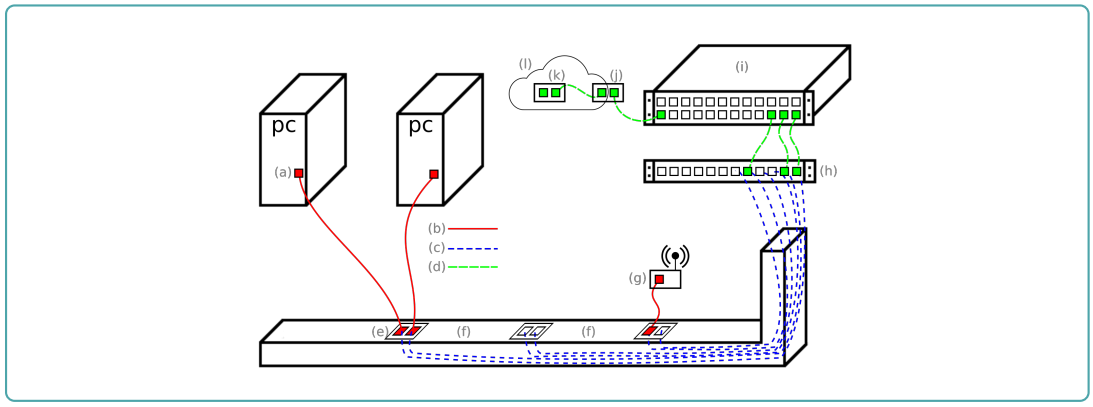
Chapter 14

Emerging Technologies

AI includes systems for perception, language, prediction or planning. Machine learning uses data to fit patterns. Generative AI produces text, images, code or audio. Outputs can be inaccurate or biased; evaluate them and keep human oversight for consequential decisions. Cloud computing offers on-demand computing, storage, databases and software. IaaS provides infrastructure, PaaS a managed platform and SaaS a ready-to-use application. Networks and routing provide the connectivity between clients, services and data centres.



Network segmentation can separate workloads and reduce unnecessary traffic; source and licence are listed in IMAGE-CREDITS.txt. Internet of Things devices combine sensors, software and connectivity. Edge computing processes data near its source, potentially reducing latency.



A network diagram helps explain how devices and infrastructure are connected; source and licence are listed in IMAGE-CREDITS.txt. Cloud and desktop environments should be configured with access controls, updates and recovery plans; an attractive interface alone does not establish security or reliability. Blockchain is a shared ledger maintained using a consensus mechanism; it is not automatically private or suitable for every database problem. Augmented reality overlays digital information on the physical world; virtual reality immerses users in a simulated environment. Robotic process automation performs repetitive rule-based tasks but needs controls for exceptions and auditability.

Review Questions

1. Distinguish AI, ML and generative AI.
2. Compare IaaS, PaaS and SaaS.
3. When might edge computing help?
4. Give one limitation of blockchain.
5. Name a responsible AI safeguard.

Chapter 15

Social Impact, Ethics and Cyber Safety

Technology can expand access but may exclude people due to cost, connectivity, language, disability or limited digital skills. Accessible design considers keyboard use, contrast, captions and assistive technology. Collect only necessary personal data, restrict access and protect it in storage and transfer. Cyber safety includes unique passwords, multifactor authentication, timely updates, backups and caution with unexpected links. Phishing attempts to trick people into disclosing information or running unsafe actions. Respect copyright, software licences, privacy and attribution. Consider energy use, repairability, device lifespan and responsible e-waste disposal.

Review Questions

1. Define digital inclusion.
2. List five principles for responsible data handling.
3. Give common phishing warning signs.
4. Why do software licences and image credits matter?
5. Suggest three sustainable computing practices.

Chapter 16

Integrated Practical Project

Design a small student learning-record system using fictional data only. Write functional requirements for adding a student, recording a score and producing a class summary. Add non-functional requirements for access control, clarity and recovery. Design tables with primary and foreign keys, create test data and query averages and counts. Use Python to process results and create a labelled chart. Separate database access, validation and presentation into functions. Prepare normal, boundary and invalid-input tests, record expected and actual results, document setup and limitations, and identify privacy risks.

Review Questions

1. Draw a basic entity relationship model.
2. Write two functional and two non-functional requirements.
3. Propose five test cases.
4. Explain how Python and SQL work together.
5. Identify two privacy risks and mitigations.